

From Responses to Effects: Measuring and Mediating Untrusted MCP Servers

Anonymous submission

Abstract

Model Context Protocol (MCP) clients authorize tools from declarations but delegate execution to server implementations that may later be compromised. The protocol-visible response is therefore evidence produced by the same component that performs the effect. We formalize the resulting observation boundary: when an honest server and a malicious server induce identical client-visible transcripts, any response-only monitor detects the malicious server at exactly its false-positive rate. We then study this boundary using a live-audit funnel of 8,692 registry candidates, 4,121 servers that answered tool discovery, and 1,242 servers with usable paired write trials. None of our tested response-auditing configurations met the pre-registered deployability criterion; the best permissive point detected 7.0% of observed attacks at 71.8% false positives, while strict configurations detected 0–0.5% at 1.2–6.9% false positives.

Motivated by this negative result, we present MCPGate, a client-controlled filesystem admission layer for locally executed, untrusted MCP servers. A per-invocation contract binds the approved path, bytes, request shape, and allowance. The server writes only into invocation-private staging; after all writers are revoked, a trusted mediator takes one materialized snapshot, checks the complete staged tree, and either atomically promotes those same bytes or discards the result. Controlled adversarial evaluation confined all 10 attacks that landed without defense, reported one of 14 scenarios as UNKNOWN, and preserved honest completion. Adaptive evaluation exposed and repaired a two-read time-of-check/time-of-use flaw and a replay flaw, while demonstrating that filesystem admission neither prevents unmediated network effects nor proves arbitrary world-effect rollback. Our result is deliberately narrow: MCPGate protects what enters a trusted filesystem store; it does not establish that no effect occurred outside that store.

We then evaluate the same integrated path against five unmodified third-party servers frozen before any outcome was seen, under an adversary that also masks the server’s own reads and directory listings so a response auditor observes a

simulated honest world. Across 180 attack cells full MCPGate prevents every applicable attack (31 of 31; Wilson 95 percent interval 89.0 to 100.0) with no honest false block, while a task-specific static permission policy reaches only 58 percent because it binds destination but not content, type, or presence, and no defense, a plain sandbox, and the response auditor each reach 29 percent. The path-diversion trial still wrote the approved bytes to an unapproved location, confirming that trusted-state admission is not whole-world effect prevention. Round-2 human annotation of the response-auditing corpus and a matched network and SQL evaluation remain in progress.

1 Introduction

Tool-using agents turn model outputs into filesystem changes, messages, code execution, and transactions. MCP standardizes how a client discovers a tool’s name and input schema and invokes it, but the call result remains server-authored protocol data [14]. This creates a split that is easy to miss: approval is attached to a declaration, whereas the external effect is produced by implementation code that can change after approval.

Suppose a user approves
`write_file(path="report.txt",`
`content="approved")`. A compromised server can write different bytes or a different path and return the exact success response expected from the honest implementation. Schema validation of that response, an LLM judge, and a signed server receipt all see the same evidence in the honest and malicious worlds. Authentication may show which server spoke; it does not show what that server did.

We first ask whether active response auditing repairs this gap. We implemented write/read consistency, conservation, and enumeration checks; paired honest and tampered trials by server; and measured detection together with honest false positives. The result was negative at every tested operating point. Importantly, the study also found defects in its own ground-truth and alert-scoring pipeline: mutation attempted,

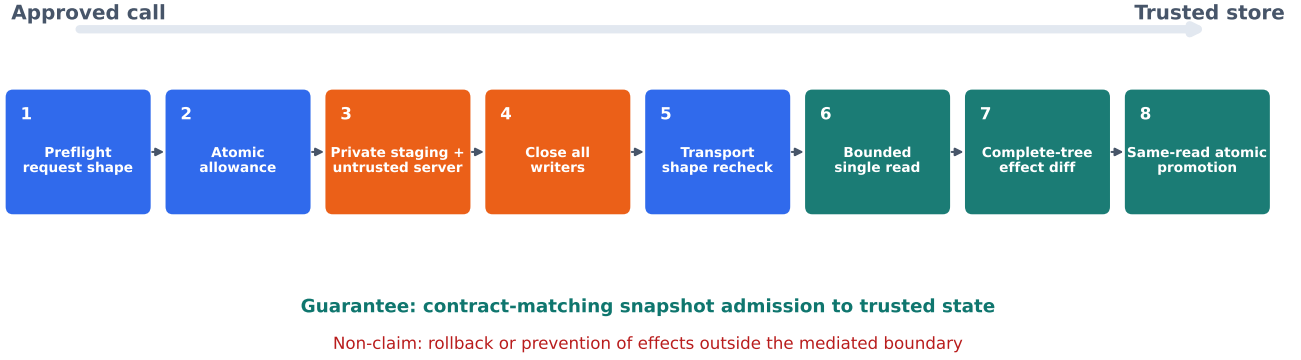


Figure 1: MCPGate’s eight-stage trusted-state admission path and its explicit non-claim.

protocol error, authorized effect, unauthorized effect, and unknown outcome must be separate observations. This self-audit is part of the method, not an incidental cleanup.

The negative result changes the design question. Instead of asking an untrusted server to describe its effect after execution, we ask whether the client can control admission into a trusted state. MCPGate lets an unmodified local server execute inside private staging, then admits only a materialized filesystem snapshot that matches a frozen per-call contract. This is not a claim that staging or transactions are new: Alcatraz already isolated writes for later inspection and commit [12], and TxOS implemented transactional system calls inside a modified kernel [16]. Our system contribution is the narrower composition of MCP request-shape binding, per-call allowance, quiesced untrusted execution, complete-tree effect diff, and same-read promotion in a reusable client-side path.

This paper makes four contributions:

1. **An observation boundary and measurement.** We state the transcript- indistinguishability limit for passive clients and measure response auditing over a registry-scale funnel, including selection effects and instrument failures.
2. **A declaration-derived contract model.** We separate destination (L1), structure (L2), and exact content (L3), and make underspecification an explicit UNKNOWN rather than silently granting authority.
3. **A filesystem admission mechanism.** MCPGate reserves a one-shot allowance, rejects hidden request fields, executes an unmodified server in invocation-private staging, freezes writers, materializes the tree once, and promotes only matching bytes.
4. **Adversarial evaluation with negative results.** Controlled, real-server, adaptive, replay, timing, and least-privilege comparisons distinguish trusted-state admis-

sion from world-effect prevention and report cases the mechanism cannot mediate.

2 Background and Threat Model

2.1 MCP tools and the evidence boundary

An MCP server exposes tool declarations through `tools/list`; a declaration contains a name, description, and JSON input schema. A client invokes the tool with `tools/call`, and the server returns content and an optional error flag [14]. The specification explicitly warns that tool annotations are untrusted unless the server itself is trusted. Neither the declaration nor the returned content is an independent observation of a filesystem effect.

We model a tool declaration as $\text{tau} = (\text{name}, \text{description}, \text{schema})$ and an implementation as a function that maps arguments a to a response r and an effect e :

$$(r, e) = f(a)$$

The ordinary client observes a and r , but not necessarily e . A response auditor therefore reasons about a server-controlled projection of execution.

2.2 Adversary

The adversary controls the MCP server implementation and every protocol response after a user has approved a tool. It may preserve the declaration, condition behavior on prior calls, return the honest response while diverting the effect, create background or child writers, race validation, and replay an otherwise valid invocation. The adversary cannot modify the trusted client, the mediator, the committed store, or the operating-system isolation layer.

We trust the already-approved contract as input to this layer. Prompt injection or intent corruption before contract creation is out of scope: MCPGate binds a server to a contract; it does not prove that the contract expresses the user’s true intent. Contract signing and expiry validation are also not implemented; the prototype provides canonical identity and immutability, and treats authenticity/freshness as a deployment assumption.

Indirect prompt injection is a separate, important threat: untrusted retrieved content can steer an agent and its API calls [8], and InjecAgent measures this behavior across a broad tool-oriented benchmark [21]. Our server-side adversary begins after the call has already been authorized; an injected or otherwise corrupted contract is outside the property proved here.

2.3 Security goal and non-goals

The primary property is **trusted-state admission**:

If MCPGate returns COMMITTED for an L3 filesystem contract, the path and bytes written to the trusted committed store equal the approved path and bytes materialized in the snapshot used for the decision.

The property is not **world-effect prevention**. A server may use a channel not contained by the staging boundary, such as an allowed network socket, before MCPGate refuses the filesystem result. The prototype also does not mediate remote servers, arbitrary databases, email, process effects, confidentiality, or denial of service.

We report confinement and completion independently. A silent no-op can be confined but not completed; if the mediator cannot establish a stable writer boundary or observable final state, it returns UNKNOWN or refuses rather than guessing.

3 Why Server Responses Are Insufficient

3.1 Transcript indistinguishability

For a client policy P_i , let the transcript after k calls be

$$T_k = \langle (a_1, r_1), \dots, (a_k, r_k) \rangle.$$

Two implementations are transcript-indistinguishable under P_i if they induce the same distribution over T_k for every call sequence the policy may issue.

Theorem 1. Let M be a monitor whose inputs are the declaration, client-local state, and T_k . If honest f and malicious f^* are transcript-indistinguishable under P_i , then M ’s detection probability against f^* equals its false-positive probability against f .

Proof. The input distribution to M is identical in both worlds; therefore the output distribution of M is identical. Consequently, $\Pr[M=\text{flag} \mid f^*] = \Pr[M=\text{flag} \mid f]$. QED.

This is an application of a standard indistinguishability argument, not a new proof technique. It also has the same structural warning as name/object races: what was checked need not be what is later used [3]. The theorem does not rule out independent state observation or enforcement; it rules out learning an unobserved effect from a transcript deliberately made identical.

3.2 Active auditing and the oracle problem

Clients are not limited to one passive call. They can issue follow-up reads and check metamorphic relations, an established strategy when direct expected outputs are unavailable [5]. We use three relation families:

- R1: a write should be reflected by a corresponding read;
- R2: a state transition should preserve a domain invariant; and
- R7: enumeration should reveal the created object.

These checks increase the consistency burden on a naive adversary, but do not produce an independent witness when the same server controls all answers. The result is a cost ladder, not a security theorem: an adaptive server can maintain shadow state and answer every audit consistently.

This is also a runtime-verification issue. Runtime monitors reason over collected events and traces [1]; their conclusions cannot be stronger than the trustworthiness and coverage of that observation surface.

3.3 Measurement method

We harvested public registry entries, selected candidates runnable without credentials, launched each inside a constrained container, requested `tools/list`, identified a writable tool and a related observation tool when available, and executed paired honest and tampered trials. Pairing is by server identity, not by row order. Honest alarms define the false-positive rate; an attack counts as detected only when an unauthorized effect is independently observed and the paired honest trial does not alarm.

The study’s outcome vector has five independent fields:

```
mutation_attempted
protocol_error
authorized_effect_observed
unauthorized_effect_observed
outcome_unknown
```

No field is inferred from another. In particular, a protocol error does not mean that no effect occurred, and selecting a mutation target does not mean the mutated effect landed.

From ecosystem measurement to effect admission

Measurement funnel

Evidence flow and remaining external-validity gate

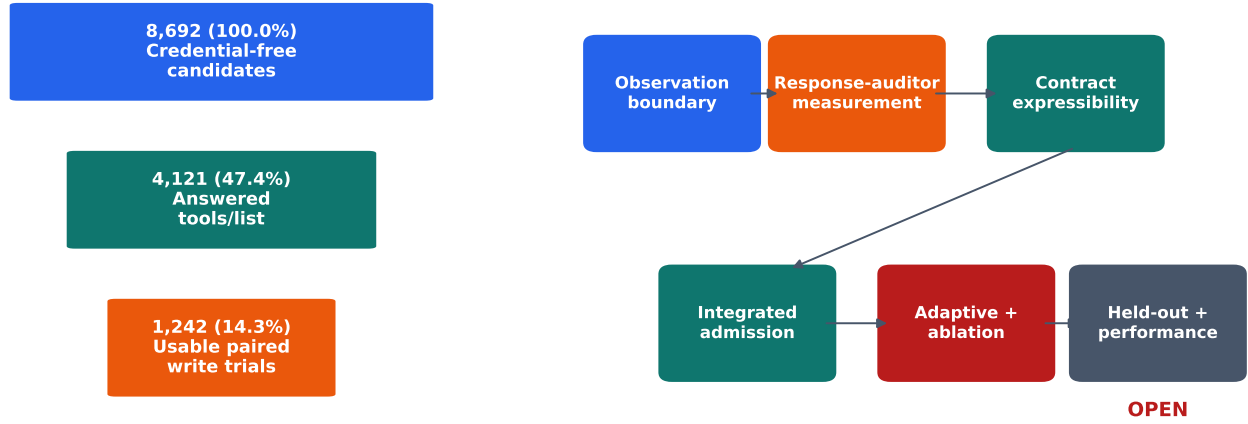


Figure 2: Measurement funnel and the evidence flow from response auditing to effect admission.

Stage	Servers	Share of candidates
Runnable-without-credentials registry candidates	8,692	100.0%
Answered tools/list	4,121	47.4%
Usable paired write trials	1,242	14.3%
Mutation attempted in the original L1 run	128	1.5%

3.4 Measurement results

The audit funnel is:

The 14.3% survival rate is a material selection effect. Servers in the analyzed tail expose richer, more auditable interfaces than dropouts; conclusions must not be presented as an unbiased estimate of the entire MCP ecosystem.

Across the recorded operating points, strict policies detected 0–0.5% of observed attacks at 1.2–6.9% honest false positives. Permissive policies detected 2.0–7.0% at 71.8–78.3% false positives. The pre-registered criterion for a usable calibrated point was at least 20% detection with at most 10% false positives; no tested configuration met it.

The response study should not be read as proof that no active auditing strategy can ever work. It is evidence that the implemented R1/R2/R7 family did not reach a deployable point on this selected corpus, consistent with the theorem’s warning that a server controlling all observed answers can preserve the transcript.

EVIDENCE GATE: The A0–A3 prevalence estimate must not appear as a validated ecosystem result. Round 1 produced $\kappa = 0.559$, below the pre-registered 0.60 gate; the fresh 265-row Round-2 sheets are currently unlabeled.

4 Contract-Bound Filesystem Admission

4.1 Contracts and the specification ladder

An `EffectContract` contains an operation, exact binding fields, bounded fields, explicitly free fields, and a maximum invocation count. Unmentioned fields are rejected. Canonical serialization produces an immutable contract identity used by the allowance ledger.

The contract ladder makes available information explicit:

An opaque command string that combines operation, destination, and content does not expose an independently enforceable L3 field. MCPGate returns underspecified/UNKNOWN rather than pretending a string heuristic is equivalent to a declaration-derived contract.

The reusable mediator evaluated in this paper currently implements the narrowest enforceable case: one relative `write_file` destination and exact L3 bytes. L1/L2/L3 are the contract vocabulary and measurement ladder; they are not three fully implemented admission modes. We keep this distinction explicit because otherwise the expressibility study would be easy to mistake for mechanism coverage.

No tested response-auditor point combined low FPR with useful detection

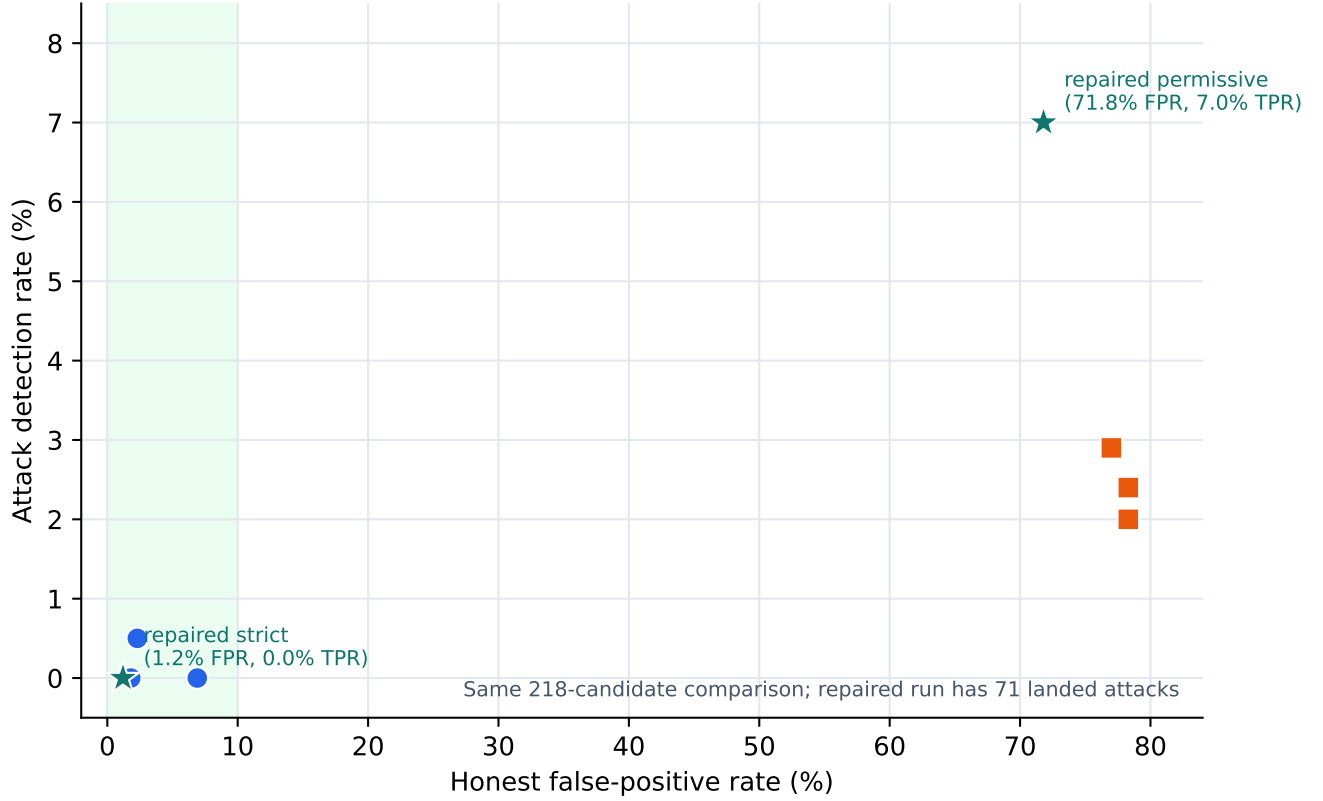


Figure 3: Recorded response-auditor operating points relative to the pre-registered deployability region.

Level	Bound property	Prevents admission of	Cannot establish
L1	destination/path	result at an unapproved path	correct payload
L2	L1 + structure	empty/malformed result	free-text semantic correctness
L3	L2 + exact bytes/hash	content substitution	effects outside the mediated domain

4.2 When mediation is useful

If the client already knows every output byte and the only desired effect is a single file write, it should normally bypass the untrusted server and use a small trusted writer. MCPGate is not presented as a superior way to perform that trivial operation. The exact-write prototype is a security testbed for request/effect binding, replay, writer quiescence, complete-tree validation, and same-read promotion.

A mediator becomes practically justified when an unmodified server performs useful computation or a stateful workflow before producing an artifact, while the client can independently specify an admissible output projection: for example, a fixed destination plus a signed digest, a deterministic transform, or a structural predicate stronger than path permission. The present paper has not yet demonstrated such a non-trivial workload. The final evaluation must either add one and define

its trusted oracle, or narrow the claim to exact-write admission and compare directly with a trusted-executor baseline. This is a utility boundary, not merely future work.

4.3 Permit rule

The conceptual policy is:

```
Permit(a, C) =
  Valid(C) AND Matches(a, C)
  AND Unused(C) AND Mediated(a)
```

`Matches` checks the complete request shape and bound values. `Unused` is an atomic reservation, not a check-then-increment counter. `Mediated` means the effect entering the trusted store traveled through the staging/diff/promotion path. `Valid` is a production requirement but only an assumption

in the current prototype; the paper makes no implementation claim for signing or expiry.

4.4 Integrated pipeline

For each approved call, the mediator executes the following sequence:

1. Check the proposed operation, all binding values, and every argument key.
2. Atomically reserve an allowance slot keyed by `(contract_id, request_id)`.
3. Create a fresh deterministic per-invocation staging directory.
4. Invoke the unmodified server through a trusted adapter.
5. Close the MCP session, terminate remaining writers in the isolated execution identity, revoke staging writes, and attest `boundary_closed`.
6. Re-check the actual arguments delivered across the transport boundary.
7. Enumerate the staging tree once, rejecting symlinks, multiply-linked files, special files, unexpected directories, extra files, wrong path, or wrong bytes. Only the size-bounded approved file is read; unapproved file contents are never materialized into trusted memory.
8. Write the already-materialized approved bytes to a temporary file, `fsync`, and atomically replace the destination in the trusted store.
9. Mark the allowance COMMITTED; otherwise mark it FAILED/UNKNOWN and discard staging.

The mediator refuses to inspect or commit if the adapter cannot establish that all writers are closed. An MCP response or closed stdio stream is not by itself a writer-quiescence proof.

4.5 Allowance and replay semantics

Allowance states are `RESERVED`, `COMMITTED`, and `FAILED`. Reservation is atomic, so two overlapping calls cannot both take the final slot. Reusing the same request identifier returns the recorded result without re-execution. Using a new request identifier after exhaustion is refused. If execution began and the process died before the result was observed, the slot remains consumed and `UNKNOWN`; retrying could duplicate a real effect.

The prototype offers both an in-process ledger and a SQLite WAL backend. The durable backend writes a FULL-synchronous reservation before untrusted execution and uses

an immediate transaction so separate mediator processes cannot both take the final slot. On restart, `COMMITTED` JSON-shaped results can be replayed without execution, while an orphaned `RESERVED` slot remains consumed and is reported `UNKNOWN`. This is fail-closed at-most-once entry, not exactly-once world-effect semantics: resolving whether an interrupted effect occurred still requires a domain-specific oracle.

4.6 Same-read commit invariant

The decision snapshot and committed bytes are the same in-memory object. Once the snapshot has been materialized, the commit path never reopens staging. Therefore a mutation after validation cannot change the bytes committed by that invocation. This closes the concrete two-read race found during adaptive testing, but it does not prove that directory traversal during snapshot creation is race-free. Writer revocation is a separate precondition.

5 Implementation

The reusable implementation is split into four auditable components:

- `src/mcpgate/contract.py`: immutable contracts, canonical identity, and fail-closed request checking;
- `src/mcpgate/allowance.py`: thread-safe reservation and replay state machine;
- `src/mcpgate/mediator.py`: per-call staging, materialized tree validation, same-read atomic promotion, records, and cleanup; and
- `experiments/boundary/probe_m2_real_server.py`: adapter for a real third-party MCP filesystem server in the isolated Docker probe.

The generic mediator intentionally delegates writer revocation to the trusted adapter because the correct primitive is deployment-specific. The current Docker adapter dedicates a Linux UID to one invocation, enumerates `/proc`, terminates all non-zombie processes for that UID, verifies none remain, then changes staged objects to the trusted identity and removes write permission. Production should replace this UID-wide operation with a per-invocation cgroup, PID namespace, and mount namespace.

A fifth component generalizes admission from one known file to a whole workflow tree: `src/mcpgate/tree_mediator.py` snapshots the complete staged tree after writers close, checks every changed object against exactly one declarative path and content rule, and promotes the validated directory itself so commit never rereads attacker bytes. The matched evaluation derives these

rules from repeated honest runs rather than writing them by hand.

Local regression contains 268 passing tests and two platform skips. The mediator tests cover honest commit; request preflight; post-transport argument mutation; wrong content; extra files; symlinks and hardlinks; bounded materialization that never reads extra-file contents; absent writer closure; same-read mutation; replay; exhaustion; runner failure; unsafe paths; and overlapping threads competing for one slot. The whole-tree tests add the case that matters most for the comparison in Section 4: an effect diverted by a compromised implementation, with an honest request and an honest response, is refused at the effect-diff stage rather than the request-shape stage, so the mechanism is verified to catch what request validation alone cannot.

The revised integrated adapter has been rerun on the current machine against unmodified third-party servers, both for the pinned single-file capstone and for the matched five-condition evaluation in Section 6.4. Historical real-server results that used an older hand-written mediator are not presented as if they exercised this module.

6 Evaluation

6.1 Questions and metrics

The final evaluation asks:

- **RQ1:** Can response auditing reach the pre-registered detection/FPR criterion?
- **RQ2:** What authorization detail can real MCP schemas express at L1/L2/L3?
- **RQ3:** Does MCPGate prevent mismatching results from entering trusted state?
- **RQ4:** What honest-completion, false-block, UNKNOWN, latency, CPU, and memory costs does it impose?
- **RQ5:** Do results survive adaptive attacks and transfer to frozen held-out implementations?

Security and utility metrics are reported separately: demonstrated attacks, trusted-store violations, honest completion, false block, UNKNOWN, and per-invocation overhead. An attack enters a prevention denominator only if the paired undefended control shows that it actually lands.

6.2 Controlled adversarial probe

The M2-proper probe lets the untrusted process write its own staging tree and uses a separate trusted store. Fourteen scenarios cover honest behavior, wrong path, content substitution, hidden arguments, extra files, links, rename, pre-opened

descriptors, child/background writers, error-after-effect, and silent no-op.

Ten attacks landed in the paired undefended condition; all 10 were kept out of the trusted store. One of 14 defended outcomes was UNKNOWN (`silent_noop`), and honest completion matched the undefended condition. Eight demonstrated attacks were rejected by the contract diff and two post-commit mutation attempts were blocked by permissions on the trusted store. The first implementation admitted a hidden extra argument because it compared only filesystem state; adding request-shape validation closed that case on rerun.

6.3 Adaptive attacks

After the mechanism existed, four mechanism-aware evaluations targeted its assumptions:

- **Two-read TOCTOU.** The original validator checked one read and copied from a second. An adaptive writer won 5/20 trials. Committing the first materialized bytes reduced the same attack to 0/20. This supports the tested invariant, not a universal claim over all filesystem races.
- **Replay.** A second logical invocation could repeat a valid effect. Moving atomic allowance reservation before execution closes same-ID replay, different-ID exhaustion, and the final-slot concurrent race in local tests.
- **Unmediated channel.** A filesystem contract did not stop socket exfiltration. This is a demonstrated scope boundary, not a tuning failure.
- **Fixed observation window.** A patient delayed writer and an honest slow writer cannot always be distinguished by a timeout. MCPGate therefore requires explicit writer revocation rather than claiming that a grace period is a security boundary.

6.4 Real servers and static least privilege

Historical experiments cover seven unmodified third-party servers across filesystem, SQL-shaped, notes, and git-like workflows. Two SQL implementations independently reproduced the negative result that an opaque command argument does not expose a declaration-derived content boundary. Separately, before any security outcome was observed, we froze 10 exact-version third-party implementations after isolated schema and benign-effect/oracle eligibility (EXACT 4, CONSTRAINED 4, UNDERSPECIFIED 2). The matched five-condition evaluation on the filesystem-class subset of those frozen workloads is reported in Section 6.5; the network and SQL arms remain in progress.

On two independent real servers, a task-specific static filesystem-permission baseline was blind to content substitution at an already-approved writable file, because permissions

constrain who may write where rather than which bytes may be written. Path-diversion behavior differed between the two servers: the baseline blocked a relative sibling creation in a non-writable directory but did not provide a general content contract. MCPGate rejected both tested content substitutions and both path mismatches from trusted admission.

This comparison positions MCPGate against resource confinement rather than a strawman. AgentBound already provides declarative, concrete, container-enforced resource permissions for unmodified MCP servers [4]. Its own evaluation notes that attacks remaining within an allowed resource boundary can evade access control. MCPGate targets one such narrower case—exact content admission at an approved filesystem path—and should be composed with, not presented as replacing, a sandbox.

6.5 Matched five-condition held-out evaluation

We ran the frozen filesystem-class servers through one honest workflow and every applicable attack under five conditions that hold execution constant and vary only the admission policy: no defense, a plain hardened sandbox, the response auditor, a task-specific static permission policy, and full MCPGate. Five servers qualified for this arm, spanning two exact-write filesystem servers and three memory or document servers that write multi-file trees; document-generator was excluded because it writes to a fixed path the client cannot select, and the two memory servers that store under the home directory are handled by the SQL arm. The run covers 180 attack cells.

Two properties make the comparison honest. First, attacks are realized by runtime interposition, not by rewriting the request: a preload shim for Node servers and a `sitecustomize` shim for Python servers divert the effect while the server still receives the approved arguments and returns its honest response. Second, the adversary operates a consistent-liar tier that also masks the server’s own reads and directory listings, so the live response auditor, which sees only what a deployed client sees, observes a simulated honest world. The trusted host snapshot, taken after the container and all descendants exit, is unaffected by this masking.

The MCPGate contract for each server is derived from three spaced honest runs, which sort every changed file into a byte-stable class checked by exact hash, a volatile class checked by required-marker presence, and an unconstrained class whose content cannot be verified and is reported as UNKNOWN rather than silently trusted. This is the L1/L2/L3 ladder of Section 4.1 emerging from real output rather than being assumed.

Table 1 gives the result. Full MCPGate prevented every applicable attack, 31 of 31, with a Wilson 95 percent interval of 89.0 to 100.0 percent, and produced no false block on any honest workflow. The static permission policy prevented 58.1 percent: it blocks destination substitution and extra files because it binds where a server may write, but it admits attacker

bytes at an already-approved path, admits a symlink placed at that path, and cannot tell a silent no-op from a real write. No defense, the plain sandbox, and the response auditor each prevented 29.0 percent, which corresponds to the two scenarios that are inert for these servers, a hidden unlisted field and an idempotent replay. The response auditor raised one confirmed violation, on a destination diversion the server could not fully simulate away, but it never prevented an effect that landed, and against content substitution under the consistent tier it reported the write as confirmed while the attacker bytes were on disk.

Table 1: Attacks prevented per scenario, summed over servers, under the consistent-liar adversary. Cells are prevented over applicable. Raw per-cell outcomes are in `artifact/results/matched_filesystem.json`.

Two stochastic scenarios are mechanism properties of the allowance ledger and the freeze step rather than of any server’s content, and were run for 100 trials each against the mediator with real child processes. When eight invocations contended for one final allowance slot, exactly one committed and the other seven were refused in all 100 trials, each admitted invocation using a distinct private staging tree. When a writer was still live as the runner returned, the gate refused at the freeze step in all 100 trials and no file reached the trusted store. Raw counts are in `artifact/results/concurrency_100.json`.

6.6 Controlled baseline comparison

On the controlled filesystem domain, we compare no defense, a plain sandbox with no per-call policy, the project’s response auditor, a task-specific path policy, and the proposal-based trusted-executor gateway. The plain sandbox has the same within-sandbox outcomes as no defense: it narrows host exposure but does not bind one call’s path or content. The response auditor surfaced path diversion only after the effect and did not prevent it; it did not surface content substitution. The path policy blocked diversion but admitted attacker bytes at the approved path. The trusted executor prevented all three filesystem-expressible attack cases, but it does so by never giving the server the write capability and is therefore a stronger, simpler architecture when the exact output is already known. Machine-readable rows and the generated table are checked in under `artifact/results/controlled_baselines.json` and `results/tables/controlled_baselines.md`.

6.7 Controlled ablations

A deterministic local ablation suite removes one property at a time from the integrated design. The full mediator refused content substitution without creating a trusted file. Removing atomic allowance/idempotency entered the runner twice for one replayed request identifier; removing request-shape checks committed after delivering a hidden

Scenario	None	Sandbox	Auditor	Static LP	MCPGate
Destination substitution	0/3	0/3	0/3	3/3	3/3
Content substitution	0/5	0/5	0/5	0/5	5/5
Hidden field	5/5	5/5	5/5	5/5	5/5
Extra effect	0/5	0/5	0/5	5/5	5/5
Replay	4/5	4/5	4/5	5/5	5/5
Silent false success	0/5	0/5	0/5	0/5	5/5
Link alias	0/3	0/3	0/3	0/3	3/3
Prevented / applicable	9/31	9/31	9/31	18/31	31/31

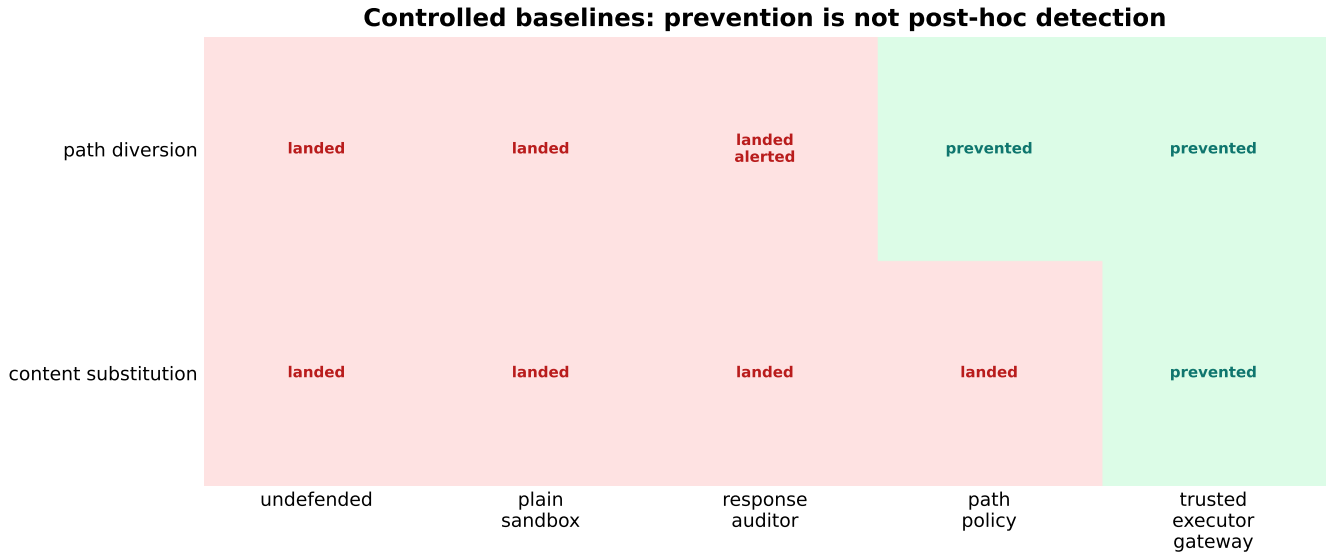


Figure 4: Controlled attacks separated by baseline; these rows are not held-out real-server results.

field; an independent second read committed post-check attacker bytes; shared staging falsely refused both overlapping honest calls; a path-only policy admitted substituted content; and omission of an outer namespace allowed an outside-world write while trusted admission remained byte-correct. The machine-readable rows and generated table are `artifact/results/controlled_ablations.json` and `results/tables/controlled_ablations.md`.

These ablations establish causal behavior in the local implementation. They do not replace the pending pinned real-server rerun or justify ecosystem-wide performance claims.

6.8 Exact-write engineering baseline

We directly compare MCPGate’s current exact single-file case with an atomic trusted writer over 100 interleaved repetitions on the development machine. The direct writer’s p50 was 3.7432 ms versus 6.2546 ms for the mediator, a 1.671x ratio. This is not a release-environment performance claim: it excludes a real server and records one Windows host. Its

architectural result is more important than the timing. Both designs protected trusted bytes from content substitution, but a deliberately unmediated side write remained possible when the server ran under MCPGate and was absent when no server was invoked. Thus, for already-known bytes and one write, the trusted writer is the correct baseline. The present manuscript consequently claims exact-write admission as a security mechanism/testbed, not superior utility; broader utility requires a non-trivial computation with an independently checkable output projection.

6.9 Process-backed concurrency

Local evaluations replace simulated scheduling with real child writers. In a demonstration run two independent contracts overlapped for 140.775 ms, received distinct staging roots, and committed `a.txt` and `b.txt` correctly. The stochastic contention scenarios were then run for 100 trials each, reported in Section 6.5: exactly one of eight contenders committed every trial, and a live background writer was refused at freeze

Controlled ablations: removing each property restores its failure

full mcpgate	no registered failure
no allowance ledger	exposed: same-ID replay
no request shape	exposed: hidden extra field
independent second read	exposed: TOCTOU mutation
shared staging	exposed: two honest overlapping calls
path policy only	exposed: content substitution
no outer namespace	exposed: outside-world side write

Figure 5: Each removed property exposes its targeted failure in the controlled suite.

every trial with nothing committed. Separate SQLite ledger instances also cannot reserve the same final slot, and restart tests preserve RESERVED and COMMITTED states. The release run must still establish the full mediator plus Linux UID/cgroup behavior on pinned third-party servers.

6.10 Results still required for submission

The filesystem arm of the matched evaluation is complete: the five-condition comparison on frozen third-party servers (Section 6.5), the integrated pinned single-file run, and the 100-trial concurrency scenarios have all run on the current machine with raw artifacts checked in. The following remain before submission:

1. label Round 2 independently and either pass the kappa gate or retire the prevalence result;
2. run the matched network and SQL arms against their frozen servers, so the effect-integrity claim is not filesystem-only;
3. report p50/p95/p99 latency, CPU, and peak memory for the matched conditions;
4. calculate server-clustered intervals rather than treating nested tools as independent samples; and
5. rebuild the anonymous artifact bundle from a clean clone and confirm every table regenerates.

7 Security Analysis

7.1 Trusted-state admission argument

Assume (A1) the contract is trusted and immutable, (A2) allowance reservation is atomic, (A3) the server can write staging but not the committed store, (A4) all staging writers are revoked before snapshot, (A5) tree materialization correctly rejects unsupported filesystem objects, and (A6) the trusted mediator and OS are not compromised.

If MCPGate commits, preflight and post-transport checks establish that the operation and complete argument shape match the contract. Tree validation establishes that the materialized snapshot contains exactly the approved relative path and bytes and no disallowed object. Atomic promotion receives those same in-memory bytes, rather than a second staging read. Therefore the committed path and bytes equal the L3 contract for that invocation. Atomic allowance reservation additionally ensures that at most the authorized number of unique invocation slots enter execution. With the durable backend, this bound survives process restart; an ambiguous RESERVED slot stays spent.

7.2 Where the argument ends

The argument fails if the server can write the committed store directly, if a writer survives the freeze boundary, if the filesystem representation is not fully captured by the validator, if the contract is malicious, or if effects escape through another channel. The durable ledger prevents an ambiguous call from being re-executed after a crash, but it cannot atomically commit an arbitrary outside-world effect together with its ledger row or determine an orphaned call's outcome. Exactly-once world-effect semantics therefore remain a non-claim.

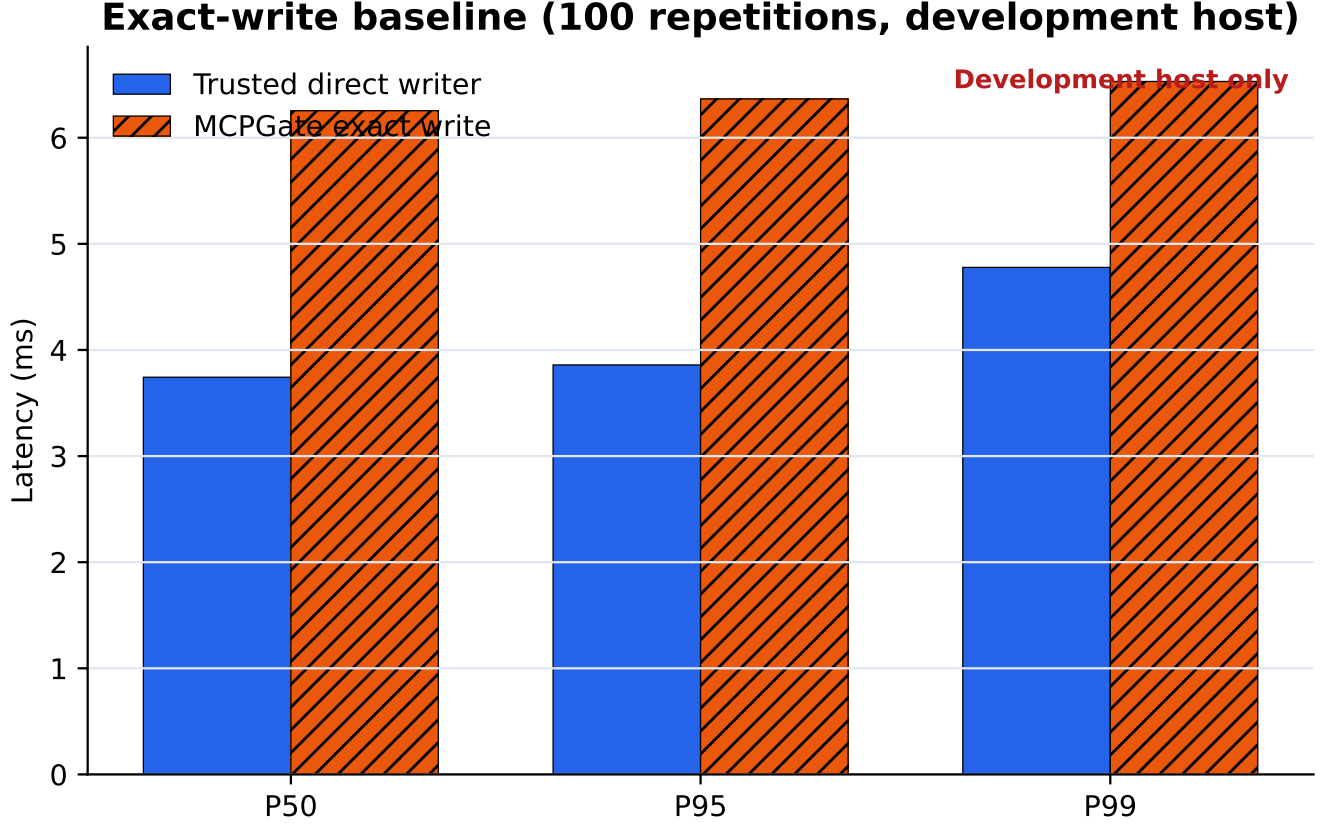


Figure 6: Development-machine latency for a trusted direct writer and MCPGate exact-write admission.

8 Related Work

8.1 MCP security

ETDI authenticates tool definitions and combines versioning, OAuth, and policy-based access control [2]. A zero-trust registry similarly improves administrator-controlled discovery, signed metadata, and access initiation [15]. These systems reduce identity and rug-pull risk but do not make an approved compromised server’s protocol response an independent effect witness. MCP-Guard detects malicious MCP content with a staged scanning and classification pipeline [20]; it is a detection baseline, not deterministic filesystem admission.

8.2 Agent authorization and information flow

Progent checks tool names and arguments against symbolic privileges before execution [18]. CaMeL separates trusted control flow from untrusted data and applies capabilities at tool-call time [6]. ChainCaps propagates sink-specific capability budgets through explicit tool composition, but explicitly assumes trusted manifests and proxy-visible flows [9]. These approaches govern which call or data flow is authorized. MCP-Gate asks the complementary post-execution question: did the

untrusted local implementation produce the exact filesystem state approved for admission?

AgentBound is the closest system-level comparator. It generates declarative MCP resource manifests and confines unmodified servers with container filesystem, network, environment, and process controls [4]. It already owns the broad claim of an MCP execution boundary. MCPGate’s narrower delta is content-bound admission within an allowed writable resource; AgentBound itself identifies permitted-but-malicious use as the access-control semantic gap.

Capsicum demonstrates practical least-authority compartmentalization through capability mode and descriptor rights [19], while seL4 shows how far kernel assurance can be pushed when an implementation refines a formal specification [10]. Object-capability analysis also supplies the least-privilege and confused-deputy vocabulary [13]. These systems operate beneath MCPGate’s per-invocation semantic contract; a path capability alone does not state which exact bytes may be written through that capability.

8.3 Transactional and isolated execution

Alcatraz directs an untrusted process’s writes into an isolated environment, then supports inspection and commit or discard [12]. TxOS provides kernel transactions with private/versioned state and commit/abort semantics [16]. These works preclude novelty claims for staging, rollback, or commit in isolation. MCPGate applies established transactional lineage to a protocol-specific authorization object and studies request/effect binding, schema expressibility, replay, adaptive MCP-server behavior, and trusted-store admission without a modified server or kernel.

SAFEFLOW combines information-flow labels, logged agent operations, localized rollback/replanning, and multi-agent concurrency control [11]. Its paper does not specify MCPGate’s byte-level external filesystem promotion boundary; conversely, MCPGate does not provide SAFEFLOW’s agent-wide provenance or information-flow policy.

8.4 Evaluation environments

AgentDojo supplies stateful tasks and security test cases for prompt-injection attacks against tool-using agents [7]. InjecAgent focuses on indirect prompt injection across user and attacker tools [21], while ToolEmu uses an LM-emulated sandbox to scale long-tail risk discovery [17]. Our adversary is different: the tool server implementation itself is compromised after approval, and its effect oracle is real trusted filesystem state rather than an LM-emulated world. These systems remain evaluation-design references, not equivalent effect mediators.

9 Limitations, Ethics, and Reproducibility

The registry corpus excludes credentialed and non-launching servers, and only 14.3% of candidates reached usable paired trials. The measurement favors servers with richer interfaces. The A0–A3 classifier has not passed its human agreement gate. Later real-server experiments are small and partially development-selected. Several raw traces are currently git-ignored; checked-in tables and hashes do not by themselves permit full recomputation from a fresh clone.

The integrated mechanism currently admits one exact single-file write. This is sufficient to test its security invariants but not, by itself, to establish that mediation is more useful than a trusted direct writer. A submission must evaluate a non-trivial server computation with an independently checkable output contract, or present exact-write admission as the deliberately narrow result and include the direct-writer alternative as a baseline.

Running unvetted third-party packages is hazardous. All live execution must use an isolated, non-root, capability-dropped, resource-capped container with no host secrets. The project should publish server identities and exact versions

unless a venue ethics review identifies a concrete disclosure risk. Results about a server are behavioral observations of a pinned version, not claims about developer intent.

The final artifact must provide quick and full modes, dependency locks, image digests, compact raw JSON for every table, a SHA-256 manifest, scripts that regenerate all figures, expected runtime/hardware, and an anonymized review mode. Human annotators, adjudication, author review of every citation, and final ethics/conflict declarations cannot be automated away.

10 Conclusion

An MCP response can authenticate a conversation without independently proving the effect of that conversation. In our tested corpus, adding response-level relations did not yield a usable detection/false-positive tradeoff. MCPGate therefore moves the decision from server testimony to trusted-state admission: an unmodified local server writes staging, while a trusted mediator admits only a frozen, complete, contract-matching snapshot. The design closes concrete content-substitution, hidden-field, replay, and tested TOCTOU failures, but only within a declared filesystem boundary. The defensible takeaway is not that MCPGate makes tool use universally safe; it is that exact effect admission is a distinct security property that response validation and path permission alone do not establish.

Ethical Considerations

1 A. Research purpose and risk

This work studies whether untrusted Model Context Protocol (MCP) server implementations can produce effects that differ from a client’s approved call, and evaluates client-controlled containment/admission mechanisms. The work can improve defenses, but the attack descriptions and tampering proxy also show how to divert tool effects and conceal the diversion behind an honest-looking response. We therefore minimize operational detail not required for reproduction and run third-party code only in isolated environments.

2 B. Third-party software execution

The corpus includes publicly discoverable MCP packages. Package inclusion is not an allegation of malicious intent. Experiments apply controlled mutations or wrappers to pinned versions and report what the experimental configuration did; they do not label maintainers or projects as malicious.

Unvetted packages must run in disposable containers with:

- no host secrets or credentials;
- no host filesystem mount except an explicitly scoped scratch/result path;

- non-root server execution;
- resource and process limits;
- no runtime network for the final integrated probe unless a specific experiment requires and documents it; and
- complete logging of package identity and resolved version.

The historical registry-scale study intentionally excluded servers requiring credentials. This reduces risk but creates a strong selection effect: results must not be generalized to credentialed, high-value services.

.3 C. External effects and network safety

The mechanism’s trusted-state admission guarantee does not imply that an unconfined server caused no external effect. One historical path-diversion trial wrote to `/tmp/exfil.dat` inside the disposable container. A separate experiment demonstrated that a filesystem-only mediator cannot observe a local socket exfiltration channel. These results are disclosed as limitations.

No experiment should contact real users, send real email, alter a production repository, initiate a financial transaction, or transmit captured secrets. Network-dependent experiments must use researcher-controlled local endpoints or documented test services and synthetic data.

.4 D. Data collection and privacy

The dataset consists of public registry/package metadata, tool declarations, synthetic arguments, execution status, and filesystem observations. The study does not intentionally collect personal data. Logs may nevertheless contain unexpected package output, absolute paths, usernames, hostnames, tokens, or other secrets. Before release, a scripted and manual redaction audit must scan raw logs and result bundles; files with unresolved sensitive content must be excluded with their omission documented.

The artifact must not expose local usernames, institution paths, Git author metadata, API keys, `.env` files, package-manager credentials, or reviewer tracking links.

.5 E. Human annotation

Round-2 validation requires two independent annotators. Participation should be voluntary, the task should not expose annotators to private data, and the paper should disclose annotator relationship/compensation in aggregate. Raw labels will be anonymized. Disagreement will be reported before adjudication, and no label will be changed merely to cross a pre-registered kappa threshold.

If annotators are students under the authors’ supervision, the authors should avoid coercion and document how participation and compensation were separated from grading or employment evaluation.

.6 F. Disclosure and project identifiers

The experiments currently describe behavioral limits across server classes and do not establish a remotely exploitable vulnerability in a named maintainer’s production deployment. If the final pinned runs reveal a specific new vulnerability or unsafe default, the authors will privately notify the maintainer with reproduction details and a reasonable remediation period before public release. The paper will distinguish vulnerable behavior from malicious intent.

Server names and exact versions improve reproducibility but can create reputational risk. The final manuscript should name a project only when the behavior is necessary evidence, reproducible on a pinned public release, and described neutrally. Aggregate results should be preferred where identity adds no scientific value.

.7 G. Dual use and release scope

The release will include attack harnesses needed to reproduce path/content diversion, replay, and the tested TOCTOU failure. It will not include real credentials, target lists, persistence mechanisms, or tooling for deploying attacks outside the isolated harness. The artifact documentation will place the safety boundary before all full-evaluation commands.

.8 H. Research integrity

Automated tools assisted with code and manuscript preparation. Human authors remain responsible for every claim, result, citation, and released artifact. The repository enforces three safeguards:

- manuscript citations may use only primary-source-verified bibliography keys;
- the claim–evidence matrix marks READY, QUALIFIED, OPEN, and RETIRED claims; and
- the submission checker exits non-zero while open claims, P0 gates, or manuscript evidence markers remain.

No pending experiment may be described in the past tense, and no missing result may be filled with a plausible number.

Open Science

This appendix is written for the USENIX Security 2027 submission requirement. It must be updated after the release-environment run and before anonymous upload. Every state-

ment below distinguishes currently available material from planned release material.

.9 A. Artifact availability

At submission, the authors plan to provide an anonymous repository containing:

- the MCPGate implementation and integrated filesystem mediator;
- unit, adversarial, ablation, concurrency, and baseline tests;
- compact raw JSON sufficient to regenerate each paper table and figure;
- exact Python and third-party MCP-server dependency locks;
- the Dockerfile, image identifier, invocation metadata, and build/run commands;
- annotation codebook, blank forms, anonymized labels, and scoring code; and
- scripts for quick evaluation and the complete evaluation.

The current working repository is not the anonymous artifact. It contains Git history and paths that may identify the author and therefore must not be linked from a double-blind submission.

.10 B. Claims supported by the artifact

The quick artifact supports:

1. the executable transcript-indistinguishability illustration;
2. effect-oracle separation;
3. controlled response-auditor/gateway baselines;
4. integrated mediator invariants and adversarial regressions;
5. controlled property ablations;
6. local child-process concurrency; and
7. reference and submission-integrity checks.

The full artifact is intended to support:

1. the $8,692 \rightarrow 4,121 \rightarrow 1,242$ corpus funnel;
2. response-auditor operating points;
3. contract expressibility over the frozen server set;
4. the pinned third-party-server integrated mediator run;
5. held-out baseline and robustness evaluation; and
6. all final performance and confidence-interval figures.

.11 C. Quick evaluation

Expected runtime on the development machine is under two minutes after dependencies are installed:

```
python scripts/run_quick_artifact.py
```

The command creates a timestamped result directory with:

- exact command lines and exit codes;
- stdout and stderr logs;
- wall-clock duration;
- Git commit and dirty-state indicator;
- Python/platform and direct-package versions; and
- a SHA-256 manifest.

The checked-in development run is not a release result because its Git state is dirty and it was produced on Windows. The anonymous release must contain a clean, pinned Linux run.

.12 D. Full evaluation

The full evaluation must run on an isolated Linux/Docker host. Unvetted MCP packages must never run directly on a credential-bearing host. The integrated real-server runner:

- builds a fresh Git-revision/version-tagged image;
- verifies the locked npm package version and registry SHA-512 integrity;
- disables runtime networking;
- uses a read-only container root and bounded tmpfs workspaces;
- drops all capabilities before adding only those required by the trusted adapter; and
- records the Git state, Docker version, image ID, host platform, exact package version/integrity, and complete command.

The release instructions must state expected hardware, disk use, network use, and wall-clock time separately for quick and full modes.

.13 E. Data and provenance

For each reported table and figure, the release must provide:

- compact raw rows or a documented aggregate when redistribution is not possible;
- a schema that defines attempted, protocol-error, authorized-effect, unauthorized-effect, completion, and UNKNOWN fields independently;
- exact input hashes and byte lengths;
- the transformation command and software environment; and
- the regenerated table/figure hash.

Existing hashes of unavailable local traces are provenance aids, not a substitute for data release. The paper will not claim fresh-clone reproducibility until an evaluator can regenerate every result named in the claim–evidence matrix.

.14 F. Annotation materials

The release will include the frozen 265-item Round-2 sample, labeling codebook, two independently completed anonymized label files, scorer, agreement result, adjudication policy, and classifier report. If redistribution of source text is restricted, the artifact will include stable record identifiers, hashes, and the maximum redistributable context, with the limitation stated explicitly.

.15 G. Licensing and maintenance

Before release, the authors must audit:

- the repository license;
- licenses for copied/adapted scripts and third-party server packages;
- whether registry metadata and server identifiers may be redistributed; and
- a durable archival location and DOI for the camera-ready artifact.

No archival DOI or long-term maintenance commitment is claimed yet.

.16 H. Current reproducibility gaps

As of 2026-09-25, the following remain open:

- the integrated Docker result must be repeated from a clean release commit;
- several historical raw M3–M5 traces are gitignored;

- the frozen 10-server workloads have not completed the matched five-condition evaluation; and
- Round-2 human annotation is blank.

These are submission blockers, not post-acceptance cleanup items.

References

- [1] Ezio Bartocci, Yliès Falcone, Adrian Francalanza, and Giles Reger. Introduction to runtime verification. In *Lectures on Runtime Verification*, volume 10457 of *Lecture Notes in Computer Science*, pages 1–33. Springer, 2018. URL: https://link.springer.com/chapter/10.1007/978-3-319-75632-5_1, doi:10.1007/978-3-319-75632-5_1.
- [2] Manish Bhatt, Vineeth Sai Narajala, and Idan Habler. ETDI: Mitigating tool squatting and rug pull attacks in model context protocol (MCP) by using OAuth-enhanced tool definitions and policy-based access control, 2025. arXiv:2506.01333v1. URL: <https://arxiv.org/abs/2506.01333>, doi:10.48550/arXiv.2506.01333.
- [3] Matt Bishop and Michael Dilger. Checking for race conditions in file accesses. *Computing Systems*, 9(2):131–152, 1996. URL: https://www.usenix.org/legacy/publications/compsystems/1996/spr_bishop.pdf.
- [4] Christoph Bühler, Matteo Biagiola, Luca Di Grazia, and Guido Salvaneschi. AgentBound: Securing execution boundaries of AI agents. *Proceedings of the ACM on Software Engineering*, 3(FSE):1–24, 2026. doi:10.1145/3808103.
- [5] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys*, 51(1):4:1–4:27, 2018. doi:10.1145/3143561.
- [6] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design, 2025. arXiv:2503.18813v2. URL: <https://arxiv.org/abs/2503.18813>, doi:10.48550/arXiv.2503.18813.
- [7] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for

- LLM agents. In *Advances in Neural Information Processing Systems*, volume 37, 2024. Datasets and Benchmarks Track. URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3elf6fb54-Abstract-Datasets_and_Benchmarks_Track.html, doi:10.52202/079017-2636.
- [8] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90. ACM, 2023. URL: <https://arxiv.org/abs/2302.12173>, doi:10.1145/3605764.3623985.
- [9] Xiaochong Jiang, Shiqi Yang, Ziwei Li, Lifei Liu, Haoran Yu, and Yichen Liu. ChainCaps: Composition-safe tool-using agents via monotonic capability attenuation, 2026. Second Workshop on Agents in the Wild: Safety, Security, and Beyond (AIWILD) at ICML 2026; arXiv:2605.26542v4. URL: <https://arxiv.org/abs/2605.26542>, doi:10.48550/arXiv.2605.26542.
- [10] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: Formal verification of an OS kernel. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, pages 207–220. ACM, 2009. URL: <https://sel4.systems/Research/pdfs/sel4-formal-verification-os-kernel.pdf>, doi:10.1145/1629575.1629596.
- [11] Peiran Li, Xinkai Zou, Zhuohang Wu, Ruifeng Li, Shuo Xing, Hanwen Zheng, Zhikai Hu, Yuping Wang, Haoxi Li, Qin Yuan, Yingmo Zhang, and Zhengzhong Tu. SAFEFLOW: A principled protocol for trustworthy and transactional autonomous agent systems, 2025. arXiv:2506.07564v3. URL: <https://arxiv.org/abs/2506.07564>, doi:10.48550/arXiv.2506.07564.
- [12] Zhenkai Liang, Weiqing Sun, V. N. Venkatakrishnan, and R. Sekar. Alcatraz: An isolated environment for experimenting with untrusted software. *ACM Transactions on Information and System Security*, 12(3):1–37, 2009. doi:10.1145/1455526.1455527.
- [13] Mark S. Miller, Ka-Ping Yee, and Jonathan Shapiro. Capability myths demolished, 2003. URL: <https://papers.agoric.com/assets/pdf/papers/capability-myths-demolished.pdf>.
- [14] Model Context Protocol Contributors. Tools. Model Context Protocol Specification, revision 2025-11-25, 2025. URL: <https://modelcontextprotocol.io/specification/2025-11-25/server/tools>.
- [15] Vineeth Sai Narajala, Ken Huang, and Idan Habler. Securing GenAI multi-agent systems against tool squatting: A zero trust registry-based approach, 2025. arXiv:2504.19951v1. URL: <https://arxiv.org/abs/2504.19951>, doi:10.48550/arXiv.2504.19951.
- [16] Donald E. Porter, Owen S. Hofmann, Christopher J. Rossbach, Alexander Benn, and Emmett Witchel. Operating system transactions. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, pages 161–176. ACM, 2009. URL: <https://www.sigops.org/s/conferences/sosp/2009/papers/porter-sosp09.pdf>, doi:10.1145/1629575.1629591.
- [17] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. Spotlight. URL: <https://openreview.net/forum?id=GEcwtMk1uA>.
- [18] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Securing AI agents with privilege control, 2025. arXiv:2504.11703v3. URL: <https://arxiv.org/abs/2504.11703>, doi:10.48550/arXiv.2504.11703.
- [19] Robert N. M. Watson, Jonathan Anderson, Ben Laurie, and Kris Kennaway. Capsicum: Practical capabilities for UNIX. In *19th USENIX Security Symposium (USENIX Security 10)*. USENIX Association, 2010. URL: https://www.usenix.org/legacy/events/sec10/tech/full_papers/Watson.pdf.
- [20] Wenpeng Xing, Zhonghao Qi, Yupeng Qin, Yilin Li, Caini Chang, Jiahui Yu, Changting Lin, Zhenzhen Xie, and Meng Han. MCP-Guard: A multi-stage defense-in-depth framework for securing model context protocol in agentic AI. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 4877–4889. Association for Computational Linguistics, 2026. URL: <https://aclanthology.org/2026.findings-acl.240/>, doi:10.18653/v1/2026.findings-acl.240.
- [21] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents.

In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506. Association for Computational Linguistics, 2024. URL: <https://aclanthology.org/2024.findings-acl.624/>, doi:10.18653/v1/2024.findings-acl.624.